

今天的内容

- 指令集架构的设计权衡(继续)
- MIPS指令集简介

指令集架构（ISA）的组成元素

■ 如何与输入/输出（I/O）设备交互

- 内存映射 I/O（Memory-Mapped I/O）
 - 将一段特定的内存地址空间映射到 I/O 设备；
 - 对 I/O 设备的读写操作通过普通的**加载（load）和存储（store）**指令完成。
- 专用 I/O 指令（Special I/O Instructions）
 - 某些体系结构（如 x86）提供专用的 I/O 指令；
 - 例如 IN 和 OUT 指令，用于通过端口（ports）与外设进行数据传输。
- 权衡与取舍（Tradeoffs）
 - 哪种方式具备更强的通用性？
 - 内存映射 I/O 允许统一的寻址与访问机制，而专用 I/O 指令可以减少地址空间的占用。

指令集架构（ISA）的组成元素

■ 特权模式（Privilege Modes）

- 用户态（User Mode）与管理态（Supervisor Mode）
- 不同权限级别下，处理器限定哪些指令可以被执行

■ 异常与中断处理（Exception and Interrupt Handling）

- 当指令执行过程中出现错误时，系统应遵循何种处理流程？
- 当外部设备向处理器发出请求时，应如何响应？
- 向量化中断（Vectored Interrupt）与非向量化中断（Non-vectored Interrupt）的区别（早期 MIPS 架构中的实现示例）

■ 虚拟内存（Virtual Memory）

- 为每个程序提供一个“完整内存空间”的抽象，使其拥有比物理内存更大的地址空间幻觉

■ 访问保护（Access Protection）

- 限制程序访问特定的内存区域或资源，以防止越权操作和系统破坏

再思考几个问题

- LC-3b 指令集体系结构 (ISA) 是否包含复杂指令?
- 一条指令的复杂程度可以达到什么水平?

复杂指令 vs. 简单指令

■ 复杂指令 (Complex Instruction)

- 一条指令完成大量工作，即同时执行多个操作。
 - 在双向链表中插入元素 (Insert in a doubly linked list)
 - 计算快速傅里叶变换 (Compute FFT)
 - 字符串拷贝 (String copy)

■ 简单指令 (Simple Instruction)

- 一条指令仅执行少量工作，是构建复杂操作的基本单元 (primitive)。
 - 加法 (Add)
 - 异或 (XOR)
 - 乘法 (Multiply)

复杂指令 vs. 简单指令

■ 复杂指令的优势

- 指令编码更密集 (Denser encoding) → 代码体积更小 (smaller code size) →
提高内存利用率、节省片外带宽 (off-chip bandwidth)、提升缓存命中率 (cache hit rate)，实现更高效的指令打包 (better packing of instructions)。
- 编译器更简单 (Simpler compiler)：由于单条指令执行的功能更多，编译器无需对小粒度指令进行大量优化。

■ 复杂指令的劣势

- 执行单元粒度较大 (Larger chunks of work) → 编译器优化空间受限，难以进行精细化 (fine-grained) 优化。
- 硬件设计更复杂 (More complex hardware) → 需要在硬件层面完成从高级操作到控制信号的转换与优化，增加了设计与实现成本。

ISA 层级的权衡：语义鸿沟（Semantic Gap）

■ ISA 应处于何种层次？——语义鸿沟（Semantic Gap）

- 更接近高级语言（High-Level Language, HLL）
 - 语义鸿沟较小（small semantic gap），指令更复杂（complex instructions）更接近硬件控制信号（Hardware Control Signals）
 - 语义鸿沟较大（large semantic gap），指令更简单（simple instructions）

■ RISC 与 CISC 体系结构的比较

- RISC（Reduced Instruction Set Computer）
精简指令集计算机，强调少而简单的指令、流水线化和编译器优化。
- CISC（Complex Instruction Set Computer）
复杂指令集计算机，提供功能丰富的指令以直接支持复杂运算。
 - FFT、快速排序（QUICKSORT）、多项式运算（POLY）、浮点运算指令（FP Instructions）
 - VAX 的 INDEX 指令：支持带边界检查的数组访问

ISA 层级的权衡：语义鸿沟 (Semantic Gap)

■ 编译器与硬件的复杂度取舍

- 简单编译器 + 复杂硬件 vs. 复杂编译器 + 简单硬件
 - 注意：指令翻译 (translation) 或间接层 (indirection) 可能改变这种权衡关系。

■ 向后兼容性负担 (Burden of Backward Compatibility)

- 为保持对旧 ISA 的兼容性，硬件设计常需保留冗余指令或微架构支持，增加实现复杂度。

■ 性能与能耗 (Performance & Energy Consumption)

- 优化责任分配：
例如在 VAX 的 INDEX 指令中，优化应由编译器完成还是由硬件承担？
- 相关考虑因素包括：
 - 指令长度 (Instruction size)
 - 代码体积 (Code size)

x86: 小语义鸿沟的实例——字符串操作

■ 字符串操作指令的特征

- 单条指令可作用于整个字符串：
 - 将任意长度的字符串从一个位置移动到另一个位置
 - 比较两个字符串

■ 实现机制：重复执行能力（Repeated Execution in ISA）

- x86 ISA 支持通过“前缀（prefix）”机制实现指令重复执行
 - 使用 **REP** 前缀（Repeat Prefix） 指定重复操作次数

■ 示例：REP MOVSB 指令

- 指令长度仅为两个字节：
 - 一个 **REP** 前缀字节 与一个 **MOVSB** 操作码字节（F2 A4）
- 隐式源寄存器与目标寄存器：源地址寄存器：**ESI**，目标地址寄存器：**EDI**
- 隐式计数寄存器**ECX** 指定字符串的长度

x86: 小语义鸿沟的实例——字符串操作

REP MOVSB (DEST SRC)

```
IF AddressSize = 16
    THEN
        Use CX for CountReg;
    ELSE IF AddressSize = 64 and REX.W used
        THEN Use RCX for CountReg; FI;
    ELSE
        Use ECX for CountReg;
FI;
WHILE CountReg ≠ 0
    DO
        Service pending interrupts (if any);
        Execute associated string instruction;
        CountReg ← (CountReg - 1);
        IF CountReg = 0
            THEN exit WHILE loop; FI;
        IF (Repeat prefix is REPZ or REPE) and (ZF = 0)
        or (Repeat prefix is REPNZ or REPNE) and (ZF = 1)
            THEN exit WHILE loop; FI;
    OD;
```

```
DEST ← SRC;
IF (Byte move)
    THEN IF DF = 0
        THEN
            (R)ESI ← (R)ESI + 1;
            (R)EDI ← (R)EDI + 1;
        ELSE
            (R)ESI ← (R)ESI - 1;
            (R)EDI ← (R)EDI - 1;
        FI;
    ELSE IF (Word move)
        THEN IF DF = 0
            (R)ESI ← (R)ESI + 2;
            (R)EDI ← (R)EDI + 2;
            FI;
        ELSE
            (R)ESI ← (R)ESI - 2;
            (R)EDI ← (R)EDI - 2;
        FI;
    ELSE IF (Doubleword move)
        THEN IF DF = 0
            (R)ESI ← (R)ESI + 4;
            (R)EDI ← (R)EDI + 4;
            FI;
        ELSE
            (R)ESI ← (R)ESI - 4;
            (R)EDI ← (R)EDI - 4;
        FI;
    ELSE IF (Quadword move)
        THEN IF DF = 0
            (R)ESI ← (R)ESI + 8;
            (R)EDI ← (R)EDI + 8;
            FI;
        ELSE
            (R)ESI ← (R)ESI - 8;
            (R)EDI ← (R)EDI - 8;
        FI;
    FI;
```

小语义鸿沟 vs. 大语义鸿沟

■ CISC 与 RISC 的对比

- CISC（复杂指令集计算机） → 采用复杂指令
 - 产生背景：早期编译器生成的代码质量“不够好”，需要硬件通过复杂指令来弥补编译器能力的不足。
- RISC（精简指令集计算机） → 采用简单指令
 - 代表人物：**John Cocke**（约1970年代中期，IBM 801 项目）
 - 目标：实现更好的编译器控制与优化能力



■ RISC 的设计动机

- 应对存储器等待（Memory Stalls）
 - 当复杂指令执行期间遇到存储器访问延迟时，处理器往往无法并行执行其他操作。
- 简化硬件设计
 - 通过减少指令复杂度，降低设计成本，提高时钟频率。
- 编译器优化（Compiler-Driven Optimization）
 - 编译器可更灵活地重排指令、挖掘细粒度并行性（fine-grained parallelism），以减少停顿（reduce stalls）。

可以高到什么程度？低到什么程度？

■ 非常大的语义鸿沟 (Very Large Semantic Gap)

- 每条指令直接指定机器中的完整控制信号集；
- 编译器直接生成控制信号，而非抽象指令；
- 代表思想：开放微码 (Open Microcode) (John Cocke, 约1970年代)；

■ 非常小的语义鸿沟 (Very Small Semantic Gap)

- 指令集体系结构 (ISA) 几乎等同于高级语言；
- 示例：
 - Java 虚拟机 (Java Machines)
 - LISP 机器 (LISP Machines)
 - 面向对象机器 (Object-Oriented Machines)
 - 基于能力的机器 (Capability-Based Machines)

ISA 演化

- 指令集体系结构（ISA）的设计不断演化，以满足不同时代的技术约束与性能需求。
- 典型历史背景
 - 片上与片外存储容量受限（Limited on-chip and off-chip memory size）
 - 编译器优化能力有限（Limited compiler optimization technology）
 - 内存带宽受限（Limited memory bandwidth）
 - 特定应用领域的专用化需求（如多媒体扩展 MMX 指令）
- 翻译机制（Translation）的引入与作用
 - 通过硬件（HW）或软件（SW）层的“指令翻译”，可在不同 ISA 间实现兼容或优化：
 - 无论上层 ISA 如何变化，底层实现可保持一致性。

指令翻译的作用 (Effect of Translation)

- 可以通过将一种 ISA 翻译为另一种 ISA 来调整语义鸿沟 (semantic gap) 之间的取舍。
 - 例如：虚拟 ISA (Virtual ISA) → 实际实现 ISA (Implementation ISA)。
 - Intel 与 AMD 的 x86 实现
 - 在硬件中将 x86 指令翻译为微操作 (micro-operations, μ ops)
 - 这些微操作对程序员不可见，通常是更简单、更细粒度的内部指令。
 - Transmeta 的 x86 实现
 - 在软件层 (Code Morphing Software) 将 x86 指令翻译为“隐藏的” VLIW (超长指令字) 指令。
- 翻译机制改变了传统 ISA 层的界限，使得同一 ISA 可拥有不同的实现策略。
 - 设计者需权衡：
 - 性能与兼容性
 - 翻译层的复杂度与执行效率

基于硬件的指令翻译

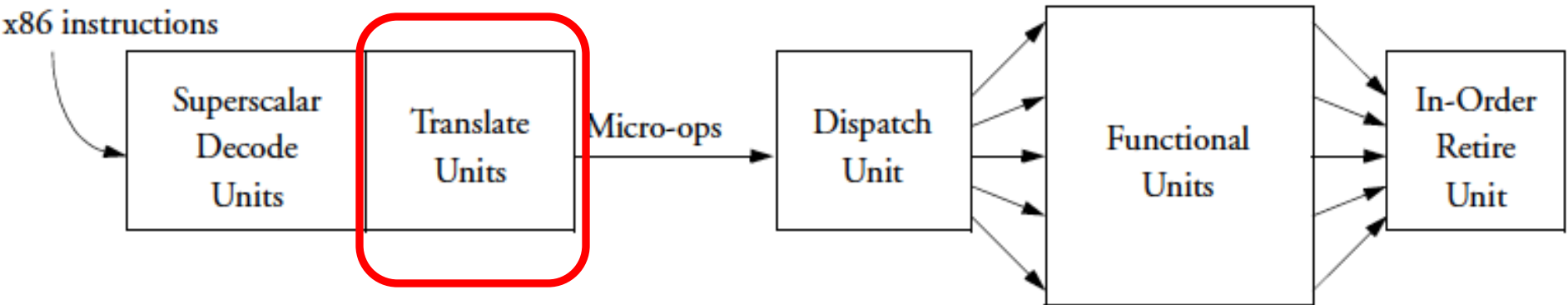


Figure 2. Conventional superscalar out-of-order CPUs use hardware to create and dispatch micro-ops that can execute in parallel.

Klaiber, “[The Technology Behind Crusoe Processors](#),” Transmeta White Paper 2000.

ISA 的权衡：指令长度

■ 固定长度指令 (Fixed Length)

- 优点：
 - 硬件中解码单条指令较为容易。
 - 支持多条指令并行解码。
- 缺点：
 - 指令中存在冗余位 (Wasted bits)，这会浪费存储空间。
 - 扩展 ISA (如何添加新指令) 变得更难。

■ 可变长度指令 (Variable Length)

- 优点：编码紧凑，有利于节省存储空间。
- 缺点：
 - 解码单条指令需要更多的逻辑处理。
 - 不易于并行解码多条指令。

■ 权衡 (Tradeoffs)

- 代码大小 (内存空间、带宽、延迟) vs. 硬件复杂性
- ISA 的可扩展性和表达能力 vs. 硬件复杂性
- 性能 vs. 能效 (小的代码 vs. 解码的简便性)

ISA 的权衡：统一解码（Uniform Decode）

■ 统一解码（Uniform Decode）定义：指令中相同的位对应相同的含义。

- 操作码（Opcode）始终位于相同位置。
- 操作数说明符、立即数等也是如此。
- 如 Alpha、MIPS、SPARC 等许多 RISC 架构。

■ 优点：

- 解码较为简单，硬件设计简化。
- 支持并行性：可在分支指令知道之前生成目标地址。

■ 缺点：

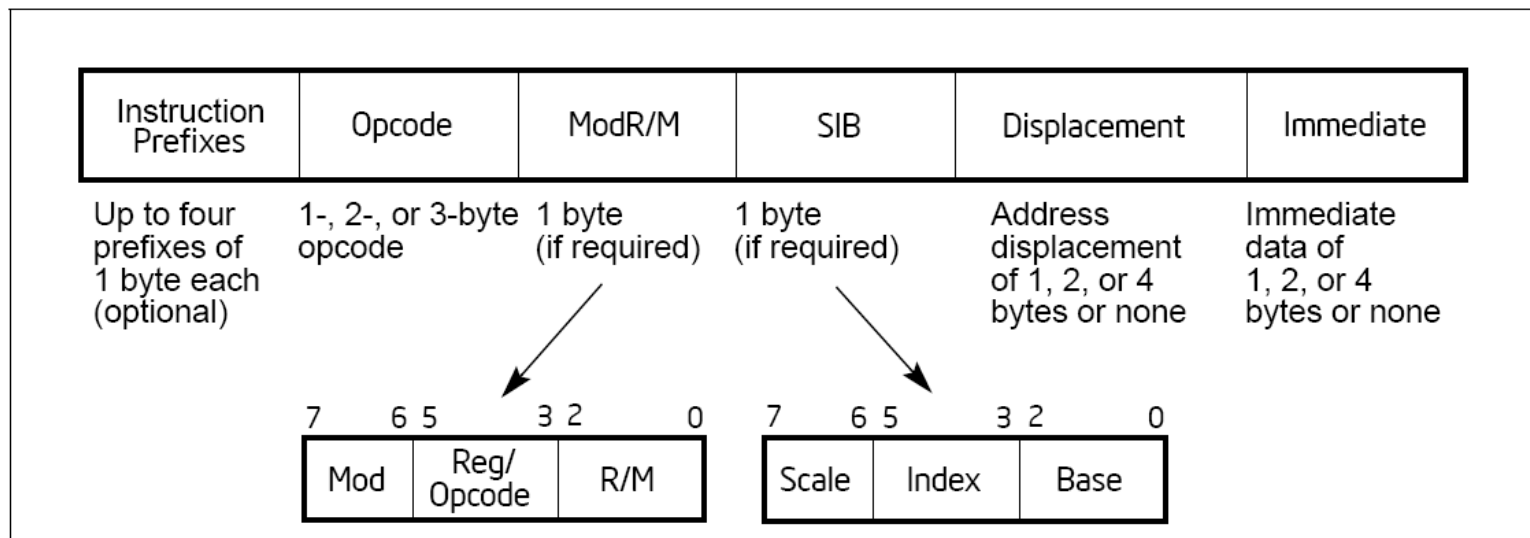
- 限制了指令格式（可能导致指令数减少？）或者浪费空间。

■ 非统一解码（Non-uniform Decode）

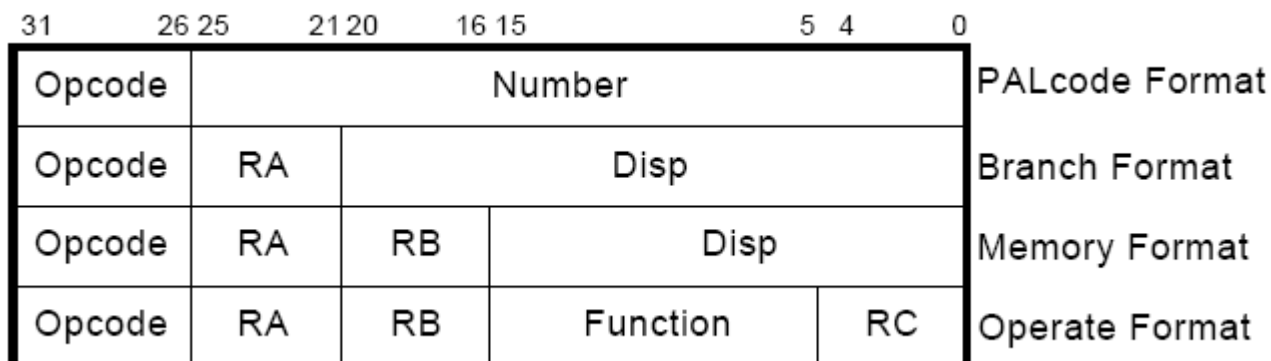
- 例如：x86 中，操作码（Opcode）可以是第 1 到第 7 字节。
 - 优点：更加紧凑和强大的指令格式。
 - 缺点：更加复杂的指令解码逻辑。

x86 与 Alpha 指令格式的对比

■ x86 指令格式

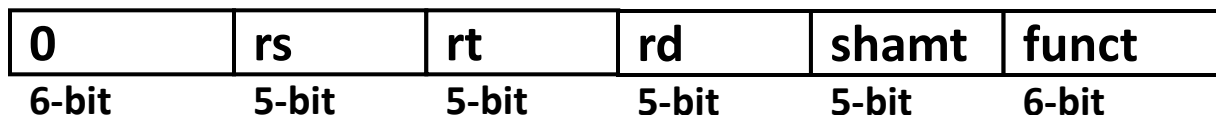


■ Alpha 指令格式



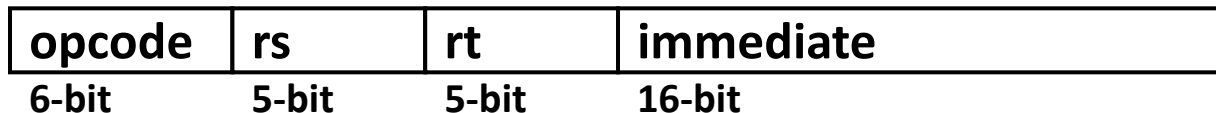
指令

- R 类型指令 (R-type) , 3 个寄存器操作数



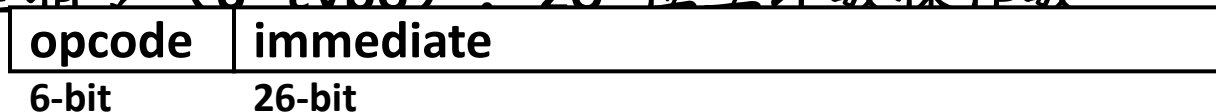
R-type

- I 类型指令 (I-type) , 2 个寄存器操作数和 16 位立即数操作数



I-type

- J 类型指令 (J-type) , 26 位立即数操作数



J-type

- 简易解码 (Simple Decoding)

- 每条指令长度为 4 字节, 无论指令格式如何
- 必须为 4 字节对齐 (PC 的最低 2 位必须为 00)
- 格式和字段容易在硬件中提取

ARM指令

3 3 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1 1 1 9 8 7 6 5 4 3 2 1 0																																	
1 0 9 8 7 6 5 4 3 2 1 0 9 8 7 6 5 4 3 2 1 0																																	
Cond	0	0	1	Opcode				S	Rn				Rd				Operand 2											<i>Data Processing / PSR Transfer</i>					
Cond	0	0	0	0	0	0	0	A	S	Rd				Rn				Rs	1	0	0	1	Rm				<i>Multiply</i>						
Cond	0	0	0	0	1	U	A	S	RdHi				RdLo				Rn				1	0	0	1	Rm				<i>Multiply Long</i>				
Cond	0	0	0	1	0	B	0	0	Rn				Rd				0	0	0	0	1	0	0	1	Rm				<i>Single Data Swap</i>				
Cond	0	0	0	1	0	0	1	0	1	1	1	1	1	1	1	1	1	1	1	0	0	0	1	Rn				<i>Branch and Exchange</i>					
Cond	0	0	0	P	U	0	W	L	Rn				Rd				0	0	0	0	1	S	H	1	Rm				<i>Halfword Data Transfer: register offset</i>				
Cond	0	0	0	P	U	1	W	L	Rn				Rd				Offset				1	S	H	1	Offset				<i>Halfword Data Transfer: immediate offset</i>				
Cond	0	1	1	P	U	B	W	L	Rn				Rd				Offset											<i>Single Data Transfer</i>					
Cond	0	1	1																					1									<i>Undefined</i>
Cond	1	0	0	P	U	S	W	L	Rn				Register List																<i>Block Data Transfer</i>				
Cond	1	0	1	L	Offset																									<i>Branch</i>			
Cond	1	1	0	P	U	N	W	L	Rn				CRd				CP#				Offset								<i>Coprocessor Data Transfer</i>				
Cond	1	1	1	0	CP Opc				CRn				CRd				CP#				CP	0	CRm				<i>Coprocessor Data Operation</i>						
Cond	1	1	1	0	CP Opc				L	CRn				Rd				CP#				CP	1	CRm				<i>Coprocessor Register Transfer</i>					
Cond	1	1	1	1	Ignored by processor																									<i>Software Interrupt</i>			
3 3 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1 1 1 9 8 7 6 5 4 3 2 1 0																																	
1 0 9 8 7 6 5 4 3 2 1 0 9 8 7 6 5 4 3 2 1 0																																	

Figure 4-1: ARM instruction set formats

关于长度与统一性 (Length and Uniformity)

- 统一解码通常与固定长度指令配合使用
 - 固定长度的指令格式，使得每条指令的解码过程相对简洁。
- 在 可变长度指令集 (ISA) 中，统一解码通常是指相同长度指令的特性。
 - 很难将统一解码作为不同长度指令的共同特性。

关于 RISC 与 CISC 的小结

■ 通常，，，，

■ RISC（精简指令集计算机）

- 简单指令
- 固定长度
- 统一解码
- 较少的寻址模式

■ CISC（复杂指令集计算机）

- 复杂指令
- 可变长度
- 非统一解码
- 较多的寻址模式

ISA 的权衡：寄存器数量

■ 影响因素：

- 寄存器地址编码所用的位数
- 存储在快速存储器（寄存器文件）中的数值数量
- 寄存器文件的大小、访问时间和功耗

■ 大量寄存器的优势：

- 允许编译器进行更好的寄存器分配（并优化程序），减少寄存器保存与恢复的次数。
- 缺点：
 - 增加指令大小。
 - 增加寄存器文件的大小。

ISA 的权衡：寻址模式

■ 寻址模式的定义：

- 寻址模式决定了如何获取操作数。
 - 寄存器寻址 (Register)
 - 立即数寻址 (Immediate)
 - 内存寻址 (Memory)：如偏移量、寄存器间接、索引、自增、自减等。

■ 更多寻址模式的优势：

- 帮助更好地支持编程构造（如数组、基于指针的访问）。
- 缺点：
 - 增加了架构设计的复杂度。
 - 对编译器的要求更高，增加了编译器的设计难度。
- 参考文献：

Wulf, "Compilers and Computer Architecture," IEEE Computer 1981

立即寻址 (Immediate Addressing)

- 在立即寻址模式中，操作数是指令中直接给出的常数值（立即数）。
- 计算方式：
 - 立即数是指令中直接提供的数值。
- 示例：
 - `addi $t0, $t1, 10`
- 这条指令将寄存器 `$t1` 的值加上立即数 10，并将结果存储到 `$t0` 中。

寄存器寻址 (Register Addressing)

- 在寄存器寻址模式中，操作数存储在寄存器中。
- 计算方式：
 - 操作数直接来自寄存器。
- 示例：
 - `add $t0, $t1, $t2`
- 这条指令将 `$t1` 和 `$t2` 寄存器的值相加，并将结果存储到 `$t0` 中。

基址寻址 (Base Addressing)

- 基址寻址使用一个寄存器作为基地址，再加上一个偏移量来计算实际地址。
- 计算方式：
 - 有效地址 = 基地址寄存器 + 偏移量
- 示例：
 - `lw $t0, 4($t1)`
- 这条指令将寄存器 `$t1` 的值作为基地址，再加上偏移量 4，得到内存地址，然后从该地址加载数据到寄存器 `$t0` 中。
- 计算：假设 `$t1` 的值为 `0x1000`，则：
 - 有效地址 = `0x1000 + 4 = 0x1004`
 - 从内存地址 `0x1004` 读取数据。

偏移量寻址 (Offset Addressing)

- 偏移量寻址类似于基址寻址，但偏移量通常是一个常数。
- 计算方式：
 - $\text{有效地址} = \text{寄存器值} + \text{偏移量}$
- 示例：
 - `sw $t0, 8($sp)`
- 这条指令将寄存器 `$t0` 的值存储到内存中，存储位置为寄存器 `$sp` 的值加上偏移量 8。
- 计算：假设 `$sp` 的值为 `0x2000`，则：
 - $\text{有效地址} = 0x2000 + 8 = 0x2008$
 - 将 `$t0` 的值存储到内存地址 `0x2008`。

PC相对寻址 (PC-relative Addressing)

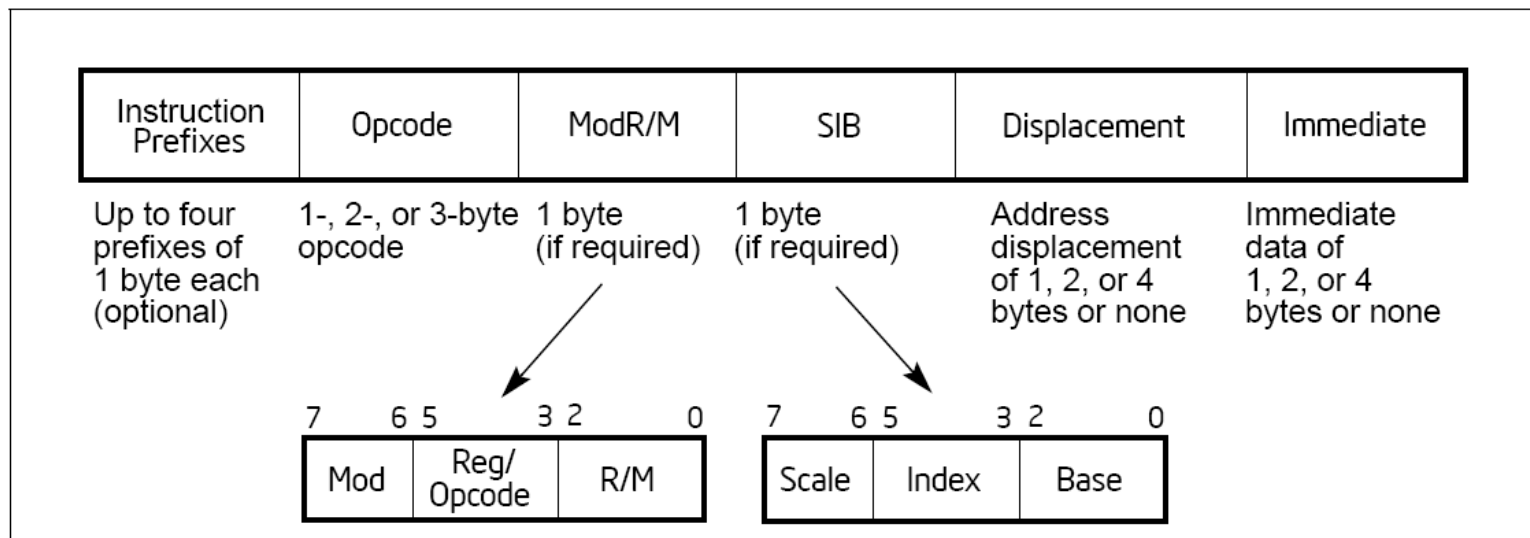
- 相对寻址模式使用当前程序计数器 (PC) 的值和指令中的偏移量来计算有效地址。
- 计算方式:
 - 有效地址 = 当前PC值 + 偏移量
偏移量是指令中给定的常数，通常是一个字 (4 字节) 为单位的偏移。
- 示例:
 - `beq $t0, $t1, label`
- 这条指令检查 `$t0` 和 `$t1` 是否相等，如果相等，则跳转到标签 `label` 所在的地址。
- 计算：假设当前 PC 值是 `0x1004`，指令中的偏移量为 `-4`（相对于当前地址的偏移）。
 - 有效地址 = $0x1004 + (-4) = 0x1000$
 - 如果 `$t0` 和 `$t1` 相等，则跳转到地址 `0x1000`。

寄存器间接寻址 (Indirect Addressing)

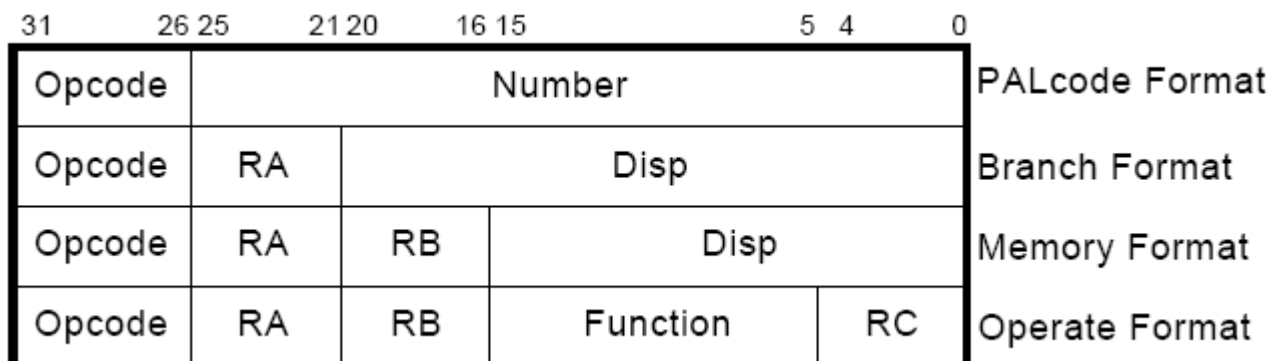
- 间接寻址模式通过寄存器存储一个地址，然后通过该地址访问数据。
- 计算方式：
 - 有效地址 = 寄存器值中的内容
寄存器存储的是一个指针，指向实际的操作数。
- 示例：
 - `lw $t0, 0($t1)`
- 这条指令将寄存器 `$t1` 中存储的地址作为指向内存的指针，从该地址加载数据到 `$t0` 中。
- 计算：
 - 假设 `$t1` 中的值为 `0x3000`，从内存地址 `0x3000` 读取数据。

x86 与 Alpha 指令格式的对比

■ x86 指令格式



■ Alpha 指令格式



X86 SIB-D寻址模式

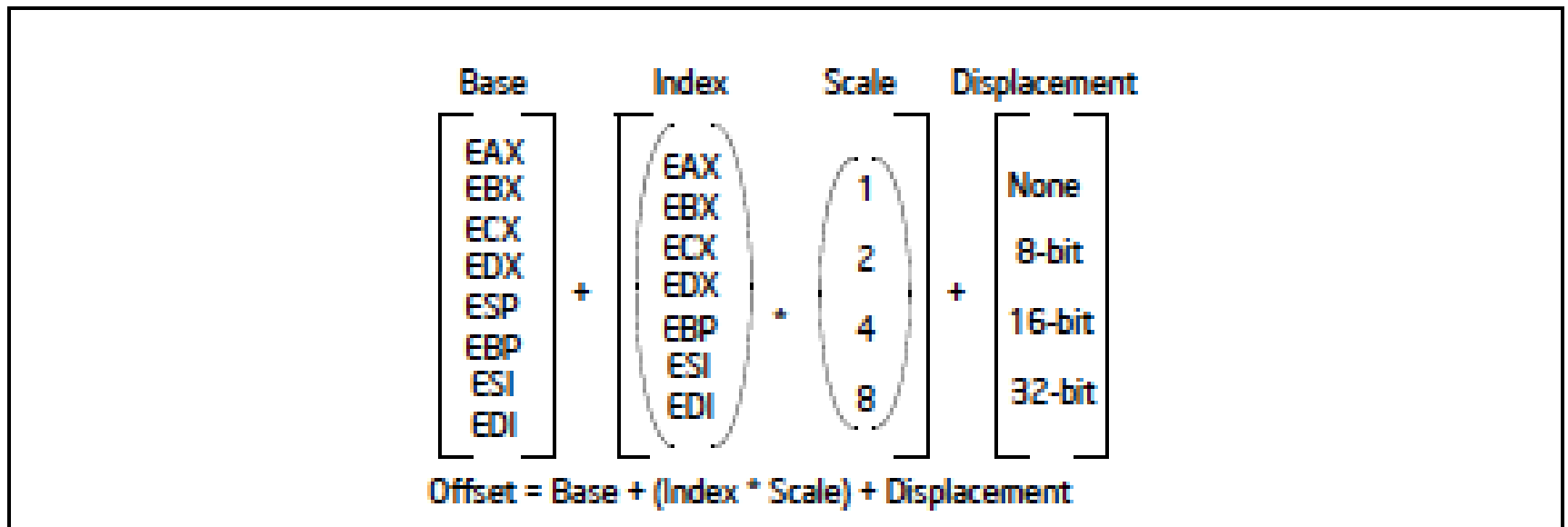


Figure 3-11. Offset (or Effective Address) Computation

X86手册：寻址模式的建议用法

Static address

Dynamic storage

The following addressing modes suggest uses for common combinations of address components.

- **Displacement** — A displacement alone represents a direct (uncomputed) offset to the operand. Because the displacement is encoded in the instruction, this form of an address is sometimes called an absolute or static address. It is commonly used to access a statically allocated scalar operand.
- **Base** — A base alone represents an indirect offset to the operand. Since the value in the base register can change, it can be used for dynamic storage of variables and data structures.
- **Base + Displacement** — A base register and a displacement can be used together for two distinct purposes:
 - As an index into an array when the element size is not 2, 4, or 8 bytes—The displacement component encodes the static offset to the beginning of the array. The base register holds the results of a calculation to determine the offset to a specific element within the array.
 - To access a field of a record: the base register holds the address of the beginning of the record, while the displacement is a static offset to the field.

An important special case of this combination is access to parameters in a procedure activation record. A procedure activation record is the stack frame created when a procedure is entered. Here, the EBP register is the best choice for the base register, because it automatically selects the stack segment. This is a compact encoding for this common function.

Arrays

Records

x86 Manual Vol. 1, page 3-22

X86手册：寻址模式的建议用法

Static arrays w/ fixed-size elements

- **(Index * Scale) + Displacement** — This address mode offers an efficient way to index into a static array when the element size is 2, 4, or 8 bytes. The displacement locates the beginning of the array, the index register holds the subscript of the desired array element, and the processor automatically converts the subscript into an index by applying the scaling factor.
- **Base + Index + Displacement** — Using two registers together supports either a two-dimensional array (the displacement holds the address of the beginning of the array) or one of several instances of an array of records (the displacement is an offset to a field within the record).
- **Base + (Index * Scale) + Displacement** — Using all the addressing components together allows efficient indexing of a two-dimensional array when the elements of the array are 2, 4, or 8 bytes in size.

2D arrays

其他 ISA 的权衡

- 条件码 vs. 无条件码
- VLIW (超长指令字) vs. 单指令
- 精确异常 vs. 非精确异常
- 虚拟内存 vs. 无虚拟内存
- 未对齐访问 vs. 对齐访问
- 硬件互锁 vs. 软件保证互锁
- 软件管理 vs. 硬件管理页面错误处理
- 缓存一致性 (硬件 vs. 软件)
-

程序员 vs. 微架构设计师

- **许多 ISA 特性旨在帮助程序员**
 - 这些特性能简化程序员的开发过程，但会增加硬件设计师的工作复杂度。
- **虚拟内存 (Virtual Memory)**
 - 与覆盖编程 (overlay programming) 相比：
 - 虚拟内存使程序员无需关心物理内存的大小。
 - 然而，硬件设计者必须处理复杂的地址映射与内存管理问题。
 - 程序员是否需要关心代码块是否能适应物理内存？
 - 但硬件设计者需承担起映射与管理责任。
- **寻址模式 (Addressing Modes)**
 - 寻址模式决定了操作数如何获取，提供了多样化的选择，程序员可根据需要选择合适的模式来优化代码的效率。
- **未对齐的内存访问 (Unaligned Memory Access)**
 - 程序员/编译器需要对齐数据：
 - 虽然未对齐访问可以增加灵活性，但它会增加硬件设计的复杂度，并可能影响性能。

MIPS: 对齐访问 (Aligned Access)

■ LW/SW 对齐限制: 4 字节对齐 (4-byte word-alignment)

- MIPS 体系结构要求每个字 (word) 必须按 4 字节对齐。
 - 不支持从不对齐的内存字节中获取数据。
 - 不支持将未对齐的字节旋转到寄存器中。

■ 为“不常见”情况提供单独的操作码

- 对于访问不对齐数据的情况, MIPS 提供了专门的操作码: LWL (Load Word Left) 和 LWR (Load Word Right)。

MSB	byte-3	byte-2	byte-1	byte-0	LSB
	byte-7	byte-6	byte-5	byte-4	

A	B	C	D
---	---	---	---

LWL rd 6(r0) →

byte-6	byte-5	byte-4	D
--------	--------	--------	---

LWR rd 3(r0) →

byte-6	byte-5	byte-4	byte-3
--------	--------	--------	--------

X86: 未对齐访问 (Unaligned Access)

■ LD/ST 指令自动对齐数据

- LD (加载) 和 ST (存储) 指令会自动对齐跨越“字”边界的数据。

■ 程序员/编译器无需关心数据存储位置

- 程序员或编译器不需要关心数据是否存储在对齐的位置 (是否在字对齐位置)。

4.1.1 Alignment of Words, Doublewords, Quadwords, and Double Quadwords

Words, doublewords, and quadwords do not need to be aligned in memory on natural boundaries. The natural boundaries for words, double words, and quadwords are even-numbered addresses, addresses evenly divisible by four, and addresses evenly divisible by eight, respectively. However, to improve the performance of programs, data structures (especially stacks) should be aligned on natural boundaries whenever possible. The reason for this is that the processor requires two memory accesses to make an unaligned memory access; aligned accesses require only one memory access. A word or doubleword operand that crosses a 4-byte boundary or a quadword operand that crosses an 8-byte boundary is considered unaligned and requires two separate memory bus cycles for access.